Faculty of Natural Science
Department of Computer Science
Kristianstad University

Course: DA562B – Grundläggande programmering I C#

Authors: Michael Christian Matei

Library system Application

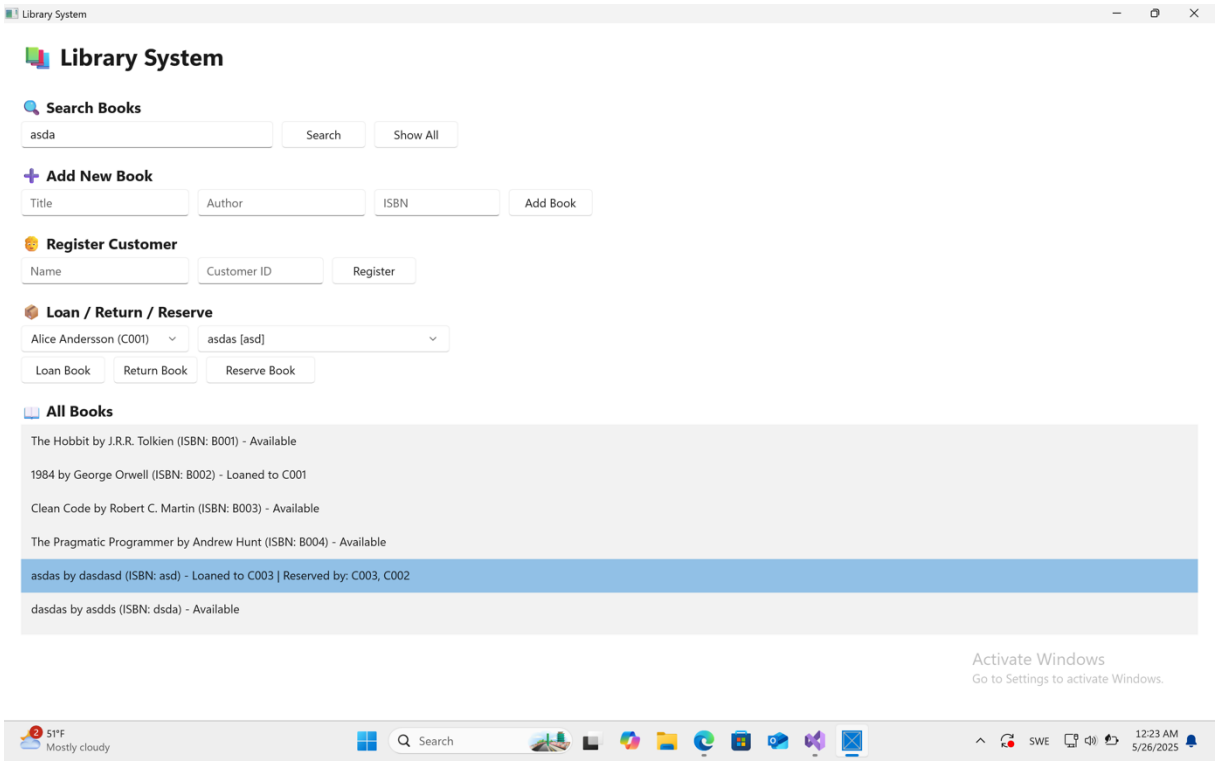


Table of Contents

1. Introduction 3

1.1 Restrictions 3

2. Requirements 4

3. Design and Implementation 4

3.1 < Book> 4

3.2 Customer 5

3.3 LibraryLogic 5

3.4 Class Diagram 6

4. Summary and Conclusion 7

1. Introduction

This project is a **Library System** developed in **C#** using **WinUI 3**. The system allows users to manage books and customers within a library — including features such as adding and removing books, registering customers, loaning and returning books, handling overdue fees, and book reservations.

The goal was to create a clean, functioning application that demonstrates good object-oriented design, GUI development, and logic handling in a real-world-inspired system.

1.1 Restrictions

While the application fulfills all the required functionality for both Version 1 and Version 2, there are a few limitations and areas for potential improvement:

- **Data is not saved permanently**: Currently, all books and customers are stored in memory during runtime. When the application closes, all data is lost. A future improvement would be to implement data persistence using file storage or a database.
- **No user login or roles**: There is no distinction between librarian and customer roles. Adding user accounts and login access would make the system more realistic for multi-user environments.
- **No overdue notifications**: Although overdue fees are calculated, there is no alert or reminder system to inform the user that a book is late. Implementing notifications or email reminders could improve usability.
- **Limited search/filter options**: The search function only supports keyword filtering by title or author. Expanding this to include filters by availability, genre, or customer ID would provide a more powerful search experience.
- **Reservations limited to 2 customers**: This is a deliberate limitation based on the assignment scope, but a more flexible queue system with waitlist management could be more scalable.
- **No reporting/export features**: Although the UI shows current loans and reservations, the system does not generate printable reports or allow exporting data to files (e.g., PDF or Excel). This could be added for professional use.

These improvements are not necessary for this school-level version, but they would be logical next steps if the system were to be expanded or deployed in a real-world setting.

2. Requirements

Table 1 below lists all the requirements for this project.

Table 1 - Requirements

Req. No	Req. Name	Req. Description
1	Add new customer	The application can register a new customer with name and unique ID.
2	Remove customer	A customer can be removed if they have no books loaned.
3	Add new book	A new book can be added with title, author, and unique ISBN.
4	Remove book	A book can be removed from the system if it is not currently loaned out.
5	Loan book	Available books can be loaned to a selected customer.
6	Return book	A customer can return a book they have borrowed.
7	Overdue fee calculation	The system calculates overdue fees (10 kr/day after a 7-day grace period).
8	Reserve book	Books currently on loan can be reserved by up to 2 other customers.
9	Track reservations	The reservation queue is saved per book and shows who reserved it.
10	Book search	Books can be searched by title or author through the GUI.
11	Show book status	Each book shows if it is available or loaned, and to which customer.
12	View customer loans	The system keeps track of which books are currently borrowed by each customer.
13	Input validation	The system prevents empty input and duplicate IDs or ISBNs.
14	Graphical interface	A modern GUI built with WinUI 3 provides access to all system features.

3. Design and Implementation

I The system is divided into three main classes: Book, Customer, and LibraryLogic. These classes follow object-oriented principles and clearly separate responsibilities. Below, each class is described in detail.

3.1 Book

The Book class represents a single book in the library.

It contains the following attributes:

- Title: the title of the book

- Author: the author of the book
- ISBN: a unique identifier for the book
- IsAvailable: indicates whether the book is available or currently loaned
- BorrowedByCustomerID: stores the ID of the customer who borrowed the book
- LoanDate: stores the date when the book was borrowed (used for fee calculation)
- ReservationQueue: a list of up to two customers who have reserved the book

Methods:

- Book(...): constructor
- GetDetails(): returns a string summary of the book's current status

This class is used by both the Customer and LibraryLogic classes to track availability and reservations.

3.2 Customer

The Customer class represents a registered library user.

Attributes:

- Name: the customer's full name
- CustomerID: a unique identifier
- LoanedBooks: a list of Book objects the customer currently has on loan

Methods:

- Customer(...): constructor
- BorrowBook(Book): adds a book to the loaned list
- ReturnBook(Book): removes a book from the loaned list
- GetLoanedBooks(): returns the list of currently borrowed books

This class interacts with Book for managing loans and is managed through LibraryLogic.

3.3 LibraryLogic

The LibraryLogic class acts as the system controller. It handles **all business logic** and connects the data to the GUI.

Attributes:

- books: a list of all books in the system
- customers: a list of all customers

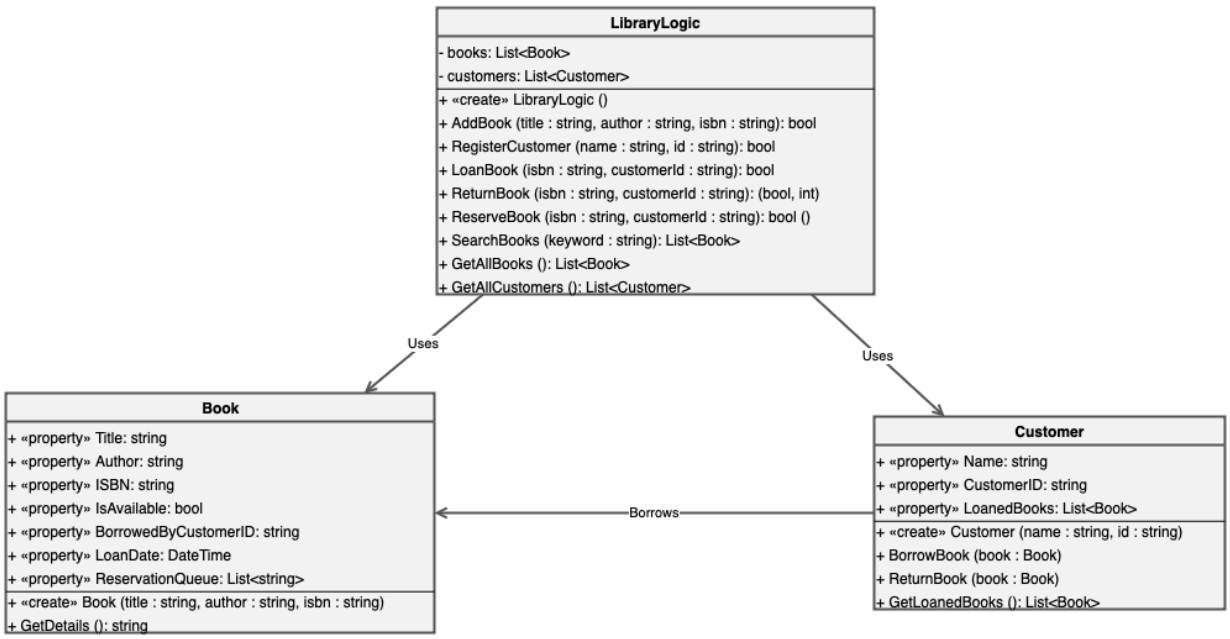
Methods include:

- AddBook(...)
- RegisterCustomer(...)
- LoanBook(...)
- ReturnBook(...) (includes overdue fee logic)
- ReserveBook(...) (with a 2-person limit)
- SearchBooks(...)
- GetAllBooks(), GetAllCustomers()

This class holds the central logic of the application and ensures consistent data handling.

3.4 Class Diagram

The following diagram illustrates how the three main classes are related:



LibraryLogic manages both Book and Customer objects. Each Customer can borrow multiple Book objects. LibraryLogic controls how loans, returns, and reservations are processed.

4. Summary and Conclusion

This project gave me valuable experience in object-oriented programming, user interface design, and logic handling. I successfully built a functional library system using C# and WinUI 3, with features like book loans, returns, overdue fee calculation, and reservation handling.

The biggest challenge was making sure only the correct customer could return a book and limiting reservations to two people. It helped me improve my code structure and logic thinking.

If I were to improve the project, I would add data saving, user logins, and more advanced search options. Overall, I'm proud of the result and feel more confident in developing real applications.